

# COMP6237 Data Mining

## Lecture 3: Covariance, EVD, PCA & SVD (Dimensionality Reduction I)

Zhiwu Huang

[Zhiwu.Huang@soton.ac.uk](mailto:Zhiwu.Huang@soton.ac.uk)

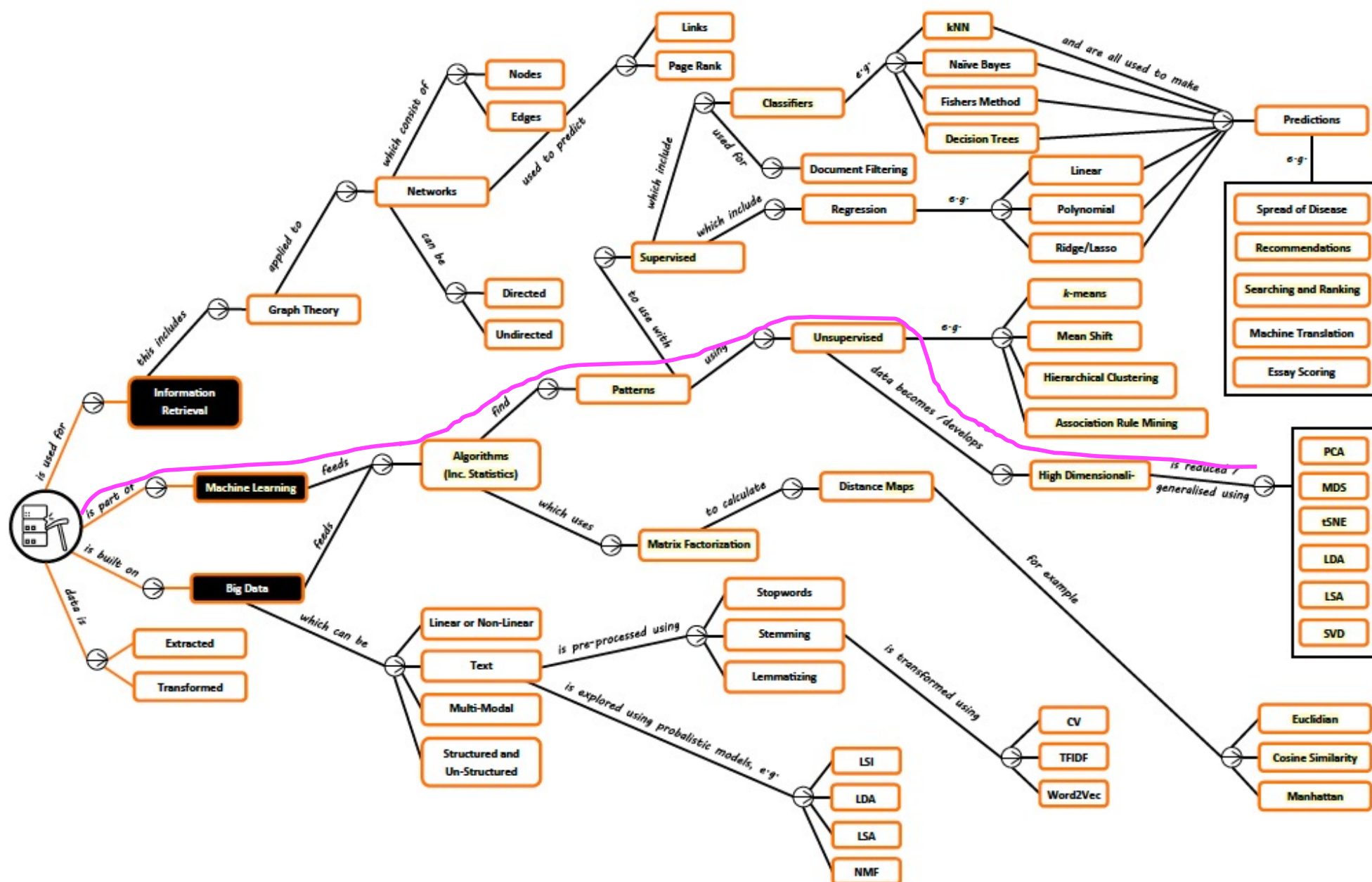
Lecturer (Assistant Professor) @ VLC of ECS  
University of Southampton

Lecture slides available here:

<http://comp6237.ecs.soton.ac.uk/zh.html>

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

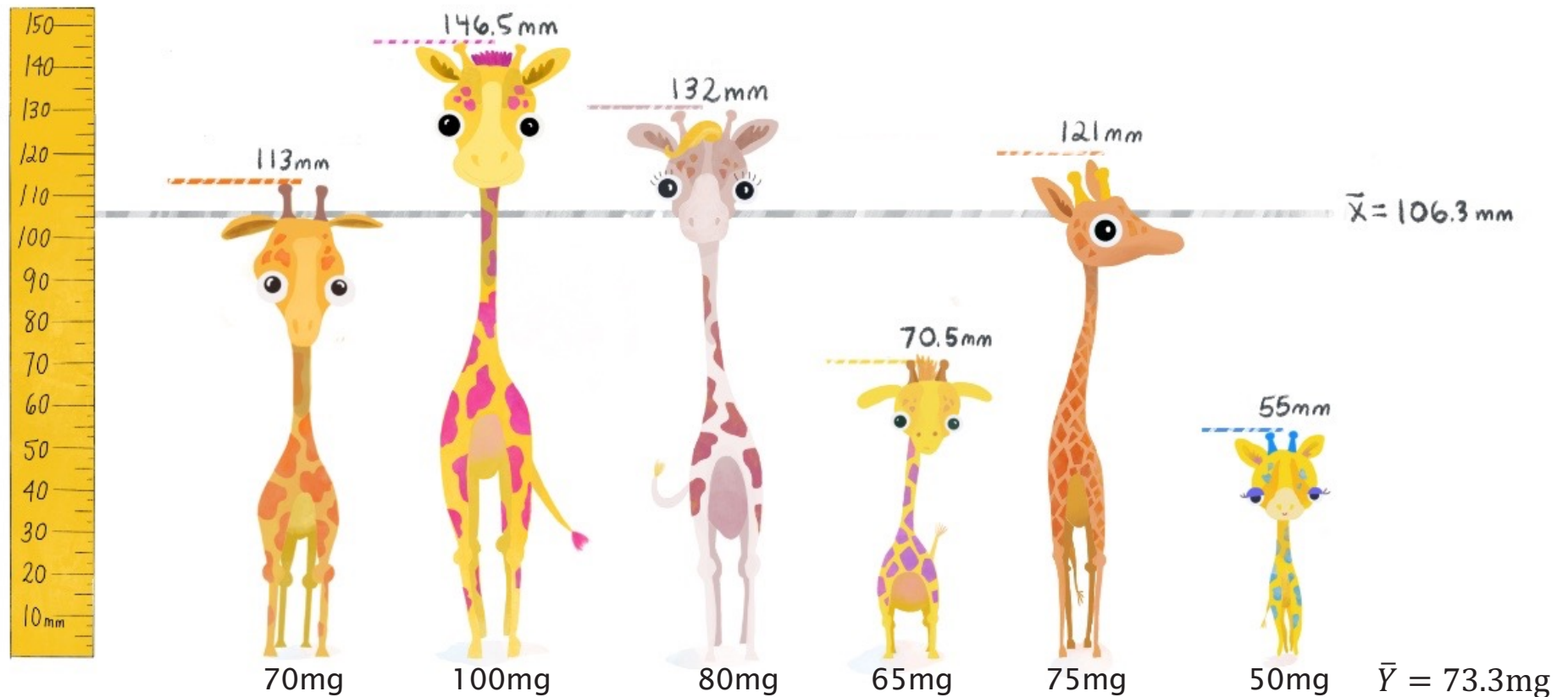
# Dimensionality Reduction I – Roadmap



# Dimensionality Reduction I – Overview (1/3)

How do we do dim Reduction?

Select those dims that represent higher data variability?



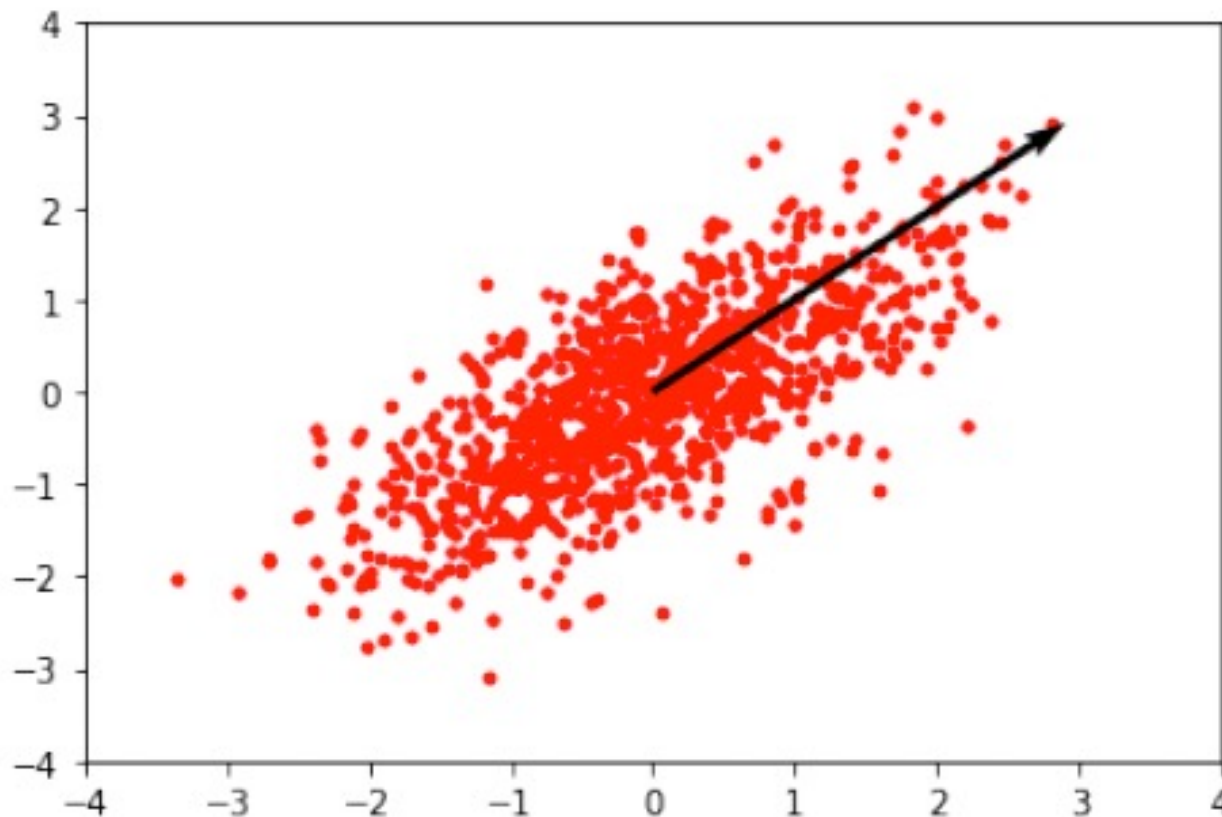
How do we measure data variability?

Range = difference between the highest and lowest values

Variance = average of squared distances from the mean

# Dimensionality Reduction I – Overview (2/3)

1st principal axis: direction of greatest variance



# Dimensionality Reduction I – Overview (3/3)

## PCA in a nutshell

3. compute covariance matrix

$$\begin{matrix} & h & u \\ h & \begin{pmatrix} 2.0 & 0.8 \end{pmatrix} \\ u & \begin{pmatrix} 0.8 & 0.6 \end{pmatrix} \end{matrix} \rightarrow \text{cov}(h,u) = \frac{1}{n} \sum_{i=1}^n h_i u_i$$

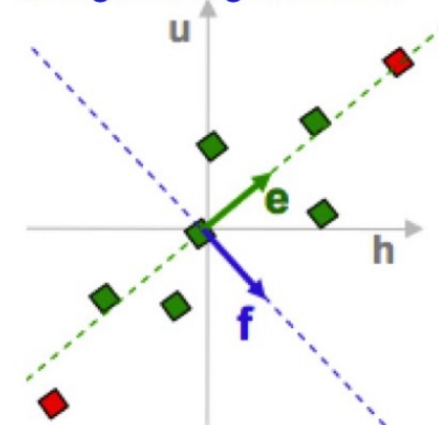
4. eigenvectors + eigenvalues

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} e_h \\ e_u \end{pmatrix} = \lambda_e \begin{pmatrix} e_h \\ e_u \end{pmatrix}$$

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} f_h \\ f_u \end{pmatrix} = \lambda_f \begin{pmatrix} f_h \\ f_u \end{pmatrix}$$

`eig(cov(data))`

5. pick  $m < d$  eigenvectors  
w. highest eigenvalues



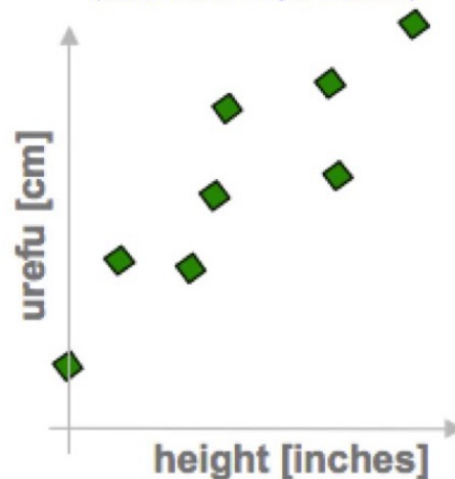
6. project data points to those eigenvectors

$$x'_e = x^T e = \sum_{j=1}^d x_{ij} e_j$$

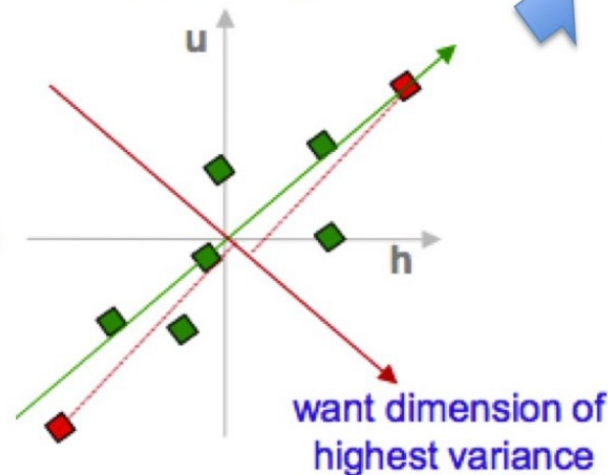
Copyright © 2014 Victor Lavrenko

1. correlated hi-d data

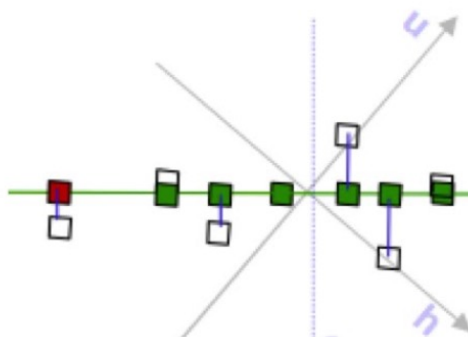
("urefu" means "height" in Swahili)



2. center the points



7. uncorrelated low-d data



# Dimensionality Reduction I - Introduction

Recap :

- ▶ Expectation and Variance
- ▶ Covariance
- ▶ Basis set
- ▶ PCA
- ▶ SVD, truncated SVD

# Dimensionality Reduction I - Expectation

A random variable takes on different values due to chance

The sample values from a single dimension of a featurespace can be considered to be a random variable

The expected value  $E[X]$  is the most likely value a random variable will take.

If we assume that the values an element of a feature can take are all equally likely then the expected value is just the mean value.

# Dimensionality Reduction I - Variance

Variance = The expected squared difference from the mean

$$E[(X - E[X])^2]$$

i.e. the mean squared difference from the mean

$$\sigma^2(x) = \frac{1}{n} = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

A measure of how spread out the data is

# Dimensionality Reduction I - Covariance

Covariance = the product of the expected difference between each feature and its mean

$$E[(x - E[x])(y - E[y])]$$

i.e. it measures how two variables change together

$$\sigma(x, y) = \frac{1}{n} \sum_{i=1}^n (x - \mu_x)(y - \mu_y)$$

When both variables are the same, covariance = variance, as

$$\sigma(x, x) = \sigma^2(x)$$

If  $\sigma^2 = 0$  then the variables are uncorrelated

# Dimensionality Reduction I - Covariance

A covariance matrix encodes how all features vary together  
For two dimensions:

$$\begin{bmatrix} \sigma(x, x) & \sigma(x, y) \\ \sigma(y, x) & \sigma(y, y) \end{bmatrix}$$

For  $n$  dimensions:

$$\begin{bmatrix} \sigma(x_1, x_1) & \sigma(x_1, x_2) & \dots & \sigma(x_1, x_n) \\ \sigma(x_2, x_1) & \sigma(x_2, x_2) & \dots & \sigma(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \sigma(x_n, x_1) & \sigma(x_n, x_2) & \dots & \sigma(x_n, x_n) \end{bmatrix}$$

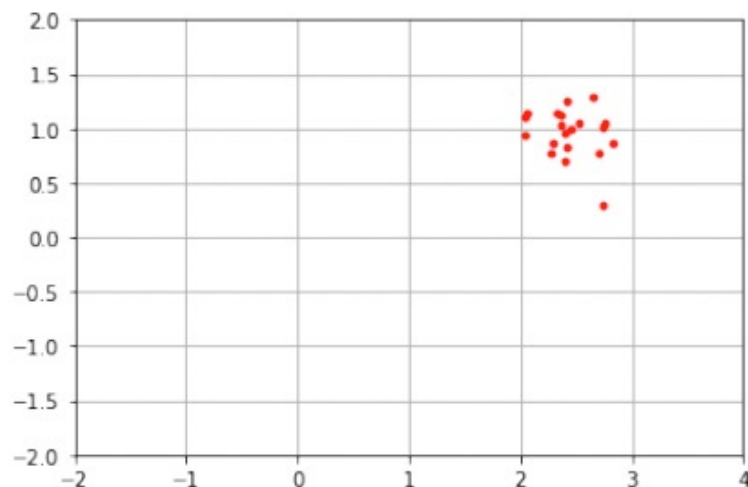
This matrix must be square symmetric

( $x$  cannot vary with  $y$  differently to how  $y$  varies with  $x$ !)

2d covariance demo

# Dimensionality Reduction I - Covariance

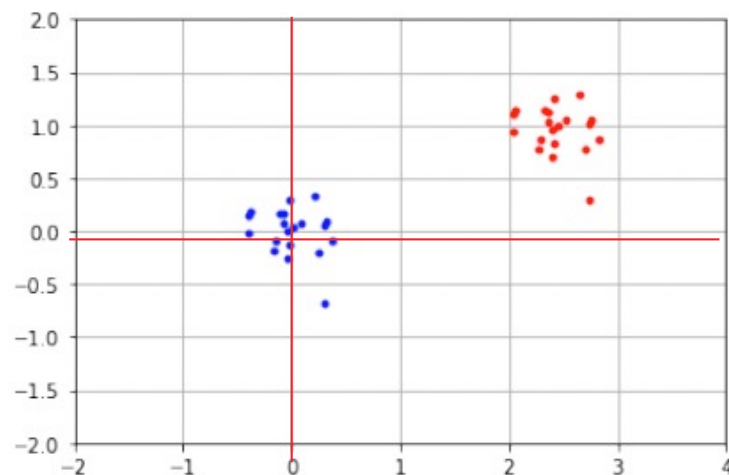
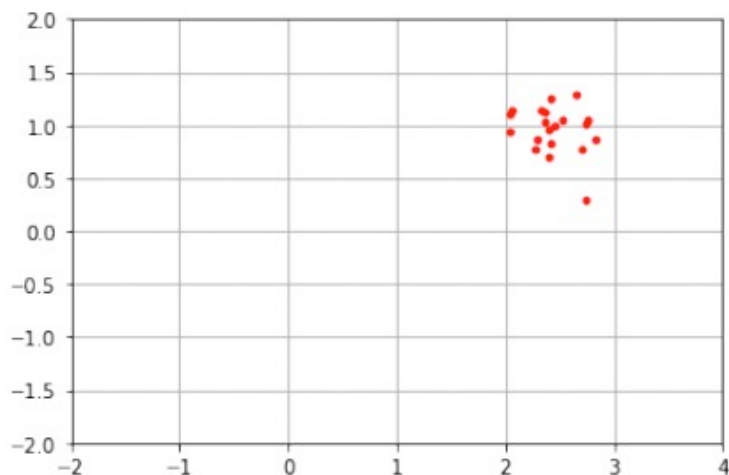
Mean Centering = subtract the mean of all the vectors from each vector  
This gives centered data, with mean at the origin



# Dimensionality Reduction I - Covariance

## Covariance computing vs. Inner product of centered data

Mean Centering = subtract the mean of all the vectors from each vector  
This gives centered data, with mean at the origin



# Dimensionality Reduction I - Covariance

## Covariance computing vs. Inner product of centered data

If you have a set of mean centred data with  $d$  dimensions, where each row is your data point:

$$Z = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \end{bmatrix}, \text{ where } x_{ij} = X_{ij} - \mu_{ij}$$

Then its inner product is proportional to the covariance matrix

$$C \propto Z^T Z$$

d-D Covariance:  $C(X) = \frac{1}{n} \sum_{i=1}^n (X_i - \mu)^T (X_i - \mu)$ , where  $X_i = [X_{i1}, \dots, X_{id}]$  is uncentred data,  $\mu = [\mu_{i1}, \dots, \mu_{id}]$ ,  $n$  is the sample number.

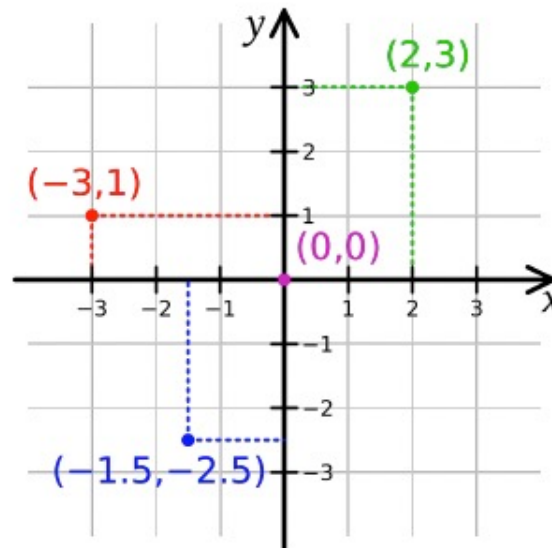
ipynb mean centering demo <https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19>

# Dimensionality Reduction I – Basis Set

In linear algebra, a basis set is defined for a space with the properties:

- ▶ They are all linearly independent
- ▶ They span the whole space

Every vector in the space can be described as a combination of basis vectors



Using Cartesian coordinates, we describe every vector as a combination of x and y directions

# Dimensionality Reduction I – Basis Set

## Eigenvectors and eigenvalues

An eigenvector is a vector that when multiplied by a matrix **A** gives a value that is a multiple of itself, i.e.:

$$\mathbf{Ax} = \lambda \mathbf{x}$$

The eigenvalue  $\lambda$  is the multiple that it should be multiplied by.

*eigen* comes from German, meaning 'Characteristic' or 'Own'

for an  $n \times n$  dimensional matrix **A** there are  $n$  eigenvector-eigenvalue pairs

# Dimensionality Reduction I – Basis Set

So if  **$A$**  is a covariance matrix, then the eigenvectors are its principal axes.

The eigenvalues are proportional to the variance of the data along each eigenvector

The eigenvector corresponding to the largest eigenvalue is the first principal axis

# Dimensionality Reduction I – Basis Set

To find eigenvectors and eigenvalues for smaller matrices, there are algebraic solutions, and all values can be found

For larger matrices, numerical solutions are found, using eigendecomposition.

Eigen - Value - Decomposition (EVD):

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$$

Where  $\mathbf{Q}$  is a matrix where the columns are the eigenvectors, and  $\mathbf{\Lambda}$  is a matrix with eigenvalues along the corresponding diagonal

Covariance matrices are real symmetric, so  $\mathbf{Q}^{-1} = \mathbf{Q}^T$  Therefore:

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$$

This diagonalisation of a covariance matrix gives the principal axes and their relative magnitudes

# Dimensionality Reduction I – Basis Set

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$$

This diagonalisation of a covariance matrix gives the principal axes and their relative magnitudes

Usually the implementation will order the eigenvectors such that the eigenvalues are sorted in order of decreasing value.

Some solvers only find the top  $k$  eigenvalues and corresponding eigenvectors, rather than all of them.

# Dimensionality Reduction I – PCA

---

**Algorithm 1:** PCA algorithm using EVD

---

**Data:**  $N$  data points with feature vectors  $X_i$   $i = 1 \dots N$

$Z = \text{meanCentre}(X)$ ;

$\text{eigVals}, \text{eigVects} = \text{eigendecomposition}(Z^T Z)$ ;

take  $k$   $\text{eigVects}$  corresponding to  $k$  largest  $\text{eigVals}$ ;

make projection matrix  $P$ ; %  $P = \text{EigVects}[1:k]$

Project data  $X_p = ZP$  in to lower dimensional space;

---

([ipynb projection demo](https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19)) <https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19>

# Dimensionality Reduction I – SVD

Eigenvalue Decomposition, EVD,  $A = Q\Lambda Q^T$  only works for symmetric matrices.

Singular value decomposition - SVD

$$A = U\Sigma V^T$$

where  $U$  and  $V$  are both different orthogonal matrices, and  $\Sigma$  is a diagonal matrix

Any matrix can be factorised this way.

Orthogonal matrices are where each column is a vector pointing in an orthogonal direction to each other,  $U^T U = U U^T = I$

# Dimensionality Reduction I – SVD

$$\begin{array}{|c|} \hline A \\ \hline m \times n \\ \hline \end{array} = \begin{array}{|c|} \hline U \\ \hline m \times p \\ \hline \end{array} \begin{array}{|c|} \hline \Sigma \\ \hline p \times p \\ \hline \end{array} \begin{array}{|c|} \hline V \\ \hline p \times n \\ \hline \end{array}$$

Where  $p$  is rank of matrix  $A$

$U$  called *left singular vectors*, contains the eigenvectors of  $AA^T$ ,  
 $V$  called *right singular vectors*, contains the eigenvectors of  $A^T A$   
 $\Sigma$  contains square roots of eigenvalues of  $AA^T$  and  $A^T A$

If  $A$  is matrix of mean centred feature vectors,  $V$  contains principal components of the covariance matrix

# Dimensionality Reduction I – SVD

---

**Algorithm 2:** PCA algorithm using SVD

---

**Data:**  $N$  data points with feature vectors  $X_i$   $i = 1 \dots N$

$Z = \text{meanCentre}(X)$ ;

$U, \Sigma, V = \text{SVD}(Z)$ ;

take  $k$  columns of  $V$  corresponding to the largest  $k$  values of  $\Sigma$ ;

make projection matrix  $P$ ;

Project data  $X_p = ZP$  in to lower dimensional space;

---

Better than using EVD of  $Z^T Z$  as:

- ▶ has better numerical stability
- ▶ can be faster

# Dimensionality Reduction I – SVD

SVD has better numerical stability:

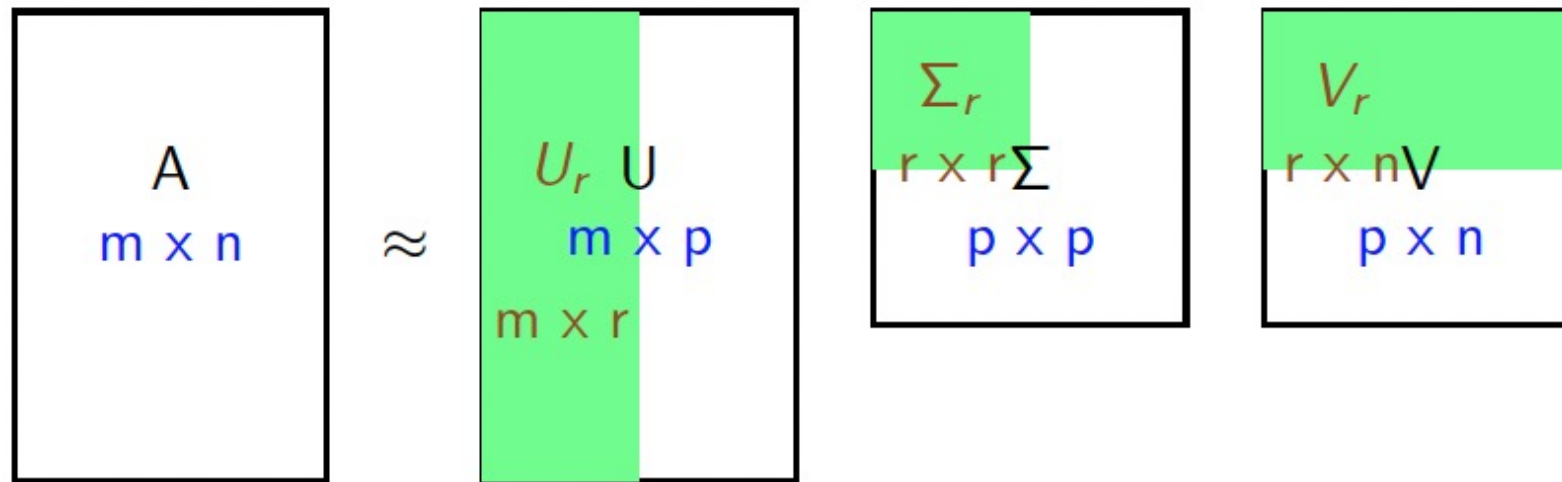
E.g. Läuchli matrix: EVD on  $X^T X$  vs. SVD on  $X$ ?

The 'Lauchli' matrix is a  $(n+1)$  by  $n$  rectangular matrix

$$X^T X = \begin{bmatrix} 1 & \epsilon & 0 & 0 \\ 1 & 0 & \epsilon & 0 \\ 1 & 0 & 0 & \epsilon \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ \epsilon & 0 & 0 \\ 0 & \epsilon & 0 \\ 0 & 0 & \epsilon \end{bmatrix} = \begin{bmatrix} 1 + \epsilon^2 & 1 & 1 \\ 1 & 1 + \epsilon^2 & 1 \\ 1 & 1 & 1 + \epsilon^2 \end{bmatrix}$$

If  $\epsilon$  is very small,  $1 + \epsilon^2$  will be counted as 1, so information is lost

# Dimensionality Reduction I – Truncated SVD



Uses only the largest  $r$  singular values (and corresponding left and right vectors)

This can give a *low rank approximation* of  $A$ ,  $\tilde{A} = U_r \Sigma_r V_r$   
 has the effect of minimising the Frobenius norm of the difference between  $A$  and  $\tilde{A}$

# Dimensionality Reduction I – Truncated SVD

SVD can be used to give a Pseudoinverse:

$$A^+ = V\Sigma^{-1}U^T$$

This is used to solve  $\mathbf{A}x = b$  for  $x$  where  $\|\mathbf{A}x - b\|_2$  is minimised  
i.e. in least squares regression  $x = \mathbf{A}^+b$

Also useful in solving homogenous linear equations  $\mathbf{A}x = 0$

SVD has also found application in:

- ▶ model based CF recommender systems
- ▶ latent factors
- ▶ image compression
- ▶ and much more..

# Dimensionality Reduction I – Summary

Covariance measures how different dimensions change together:

- ▶ Represented by a matrix
- ▶ Eigenvalue decomposition gives eigenvalue - eigenvector pairs
- ▶ The dominant eigenvector gives the principal axis
- ▶ The Eigenvalue is proportional to the variance along that axis
- ▶ The principal axes give a basis set, describing the directions of greatest variance

PCA: aligns data with its principal axes, allows dimensional reduction losing the least information by discounting axes with low variance

SVD: a general matrix factorisation tool with many uses.

# Dimensionality Reduction I – Learning Outcomes

- **LO1:** Compute basic statistics such as variance and covariance over the given data (exam)
- **LO2:** Understand the key ideas and essential steps of PCA using EVD, SVD and truncated SVD (exam).
- **LO3:** Implement the PCA algorithm for dimensionality reduction (coursework)

***Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)***

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: <https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf>*

# Dimensionality Reduction I – Textbook

## CHAPTER 7

## Dimensionality Reduction

We saw in Chapter 6 that high-dimensional data has some peculiar characteristics, some of which are counterintuitive. For example, in high dimensions the center of the space is devoid of points, with most of the points being scattered along the surface of the space or in the corners. There is also an apparent proliferation of orthogonal axes. As a consequence high-dimensional data can cause problems for data mining and analysis, although in some cases high-dimensionality can help, for example, for nonlinear classification. Nevertheless, it is important to check whether the dimensionality can be reduced while preserving the essential properties of the full data matrix. This can aid data visualization as well as data mining. In this chapter we study methods that allow us to obtain optimal lower-dimensional projections of the data.

- ▶ Data Mining and Machine Learning: Fundamental Concepts and Algorithms M. L. Zaki and W. Meira

## Chapter 11

## Dimensionality Reduction

There are many sources of data that can be viewed as a large matrix. We saw in Chapter 5 how the Web can be represented as a transition matrix. In Chapter 9, the utility matrix was a point of focus. And in Chapter 10 we

- ▶ Mining of Massive Datasets J. Leskovec *et al*